

Prompt Engineering and Structured Outputs

Design model inputs and output contracts that application code can validate.



WHY THIS MATTERS

Start from what you already know

LLM generation, Python dictionaries, JSON, functions, and validation

- 01 Explain message roles in plain language.
- 02 Use instructions and examples in a small worked example.
- 03 Complete the guided lab and verify tool calling.

CORE CONCEPT

The vocabulary of this topic

Design model inputs and output contracts that application code can validate.

message roles: Message roles distinguish trusted instructions, user requests, assistant output, and tool results.

instructions and examples: Instructions state required behavior; examples demonstrate the exact input-output pattern.

JSON schemas: A JSON schema defines required fields, types, allowed values, and nesting for machine-validated output.

tool calling: Tool calling lets a model propose a named operation and arguments; application code remains responsible for validation and execution.

FOUR CONNECTED IDEAS

How the concept works

01

message roles

02

**instructions and
examples**

03

JSON schemas

04

tool calling

Trusted instructions define task and constraints; a schema defines allowed output; application code validates before acting.

CORE CONCEPT

Read every part of the mechanism

Trusted instructions define task and constraints; a schema defines allowed output; application code validates before acting.

system message: Trusted application-level instructions.

schema: Required output fields and types.

tool call: A proposed operation and arguments.

WORKED EXAMPLE

Worked example by hand

Extract `{name, score}` from one sentence, reject a missing score, then map a validated call to a Python function.

01

Known values

02

Apply the mechanism

03

Interpret the result

RUN IT AND INSPECT THE RESULT

Map the concept to Python

```
class Result(BaseModel):  
    name: str  
    score: float = Field(ge=0, le=1)  
  
result = Result.model_validate({"name": "baseline", "score": 0.82})  
print(result.model_dump())
```

OUTPUT

```
{'name': 'baseline', 'score': 0.82}
```

Each line corresponds to a concept already explained.

QUALITY GATE

Build the complete mental model

- ☒ **Separate trusted instructions from untrusted user content and retrieved documents.**

- ☒ **Examples demonstrate behavior, but diverse tests determine whether the behavior generalizes.**

- ☒ **Validate model output against a schema before storing it or passing it downstream.**

- ☒ **Treat every tool call as an untrusted proposal requiring authorization and argument validation.**

FOUR CONNECTED IDEAS

Notebook readiness map

01

message roles

02

**instructions and
examples**

03

JSON schemas

04

tool calling

Use the concepts from this lesson to read and complete
<https://github.com/curiously/Al-Bootcamp/blob/master/13.llm-output-format.ipynb>.

RISK REVIEW

Common beginner mistakes

- 01 **Parsing important data from unconstrained prose.**

- 02 **Executing model-produced tool arguments without authorization and validation.**

GUIDED LAB

Create a prompt contract, validate structured output, and safely execute one allow-listed tool.

01

Predict

02

Run

03

Inspect

04

Explain

SESSION SYNTHESIS

Keep the system, not isolated definitions.

01 Explain message roles in plain language.

02 Use instructions and examples in a small worked example.

03 Complete the guided lab and verify tool calling.

NEXT EVIDENCE

A prompt test suite with success, malformed output, injection, unauthorized tool, and recovery cases.